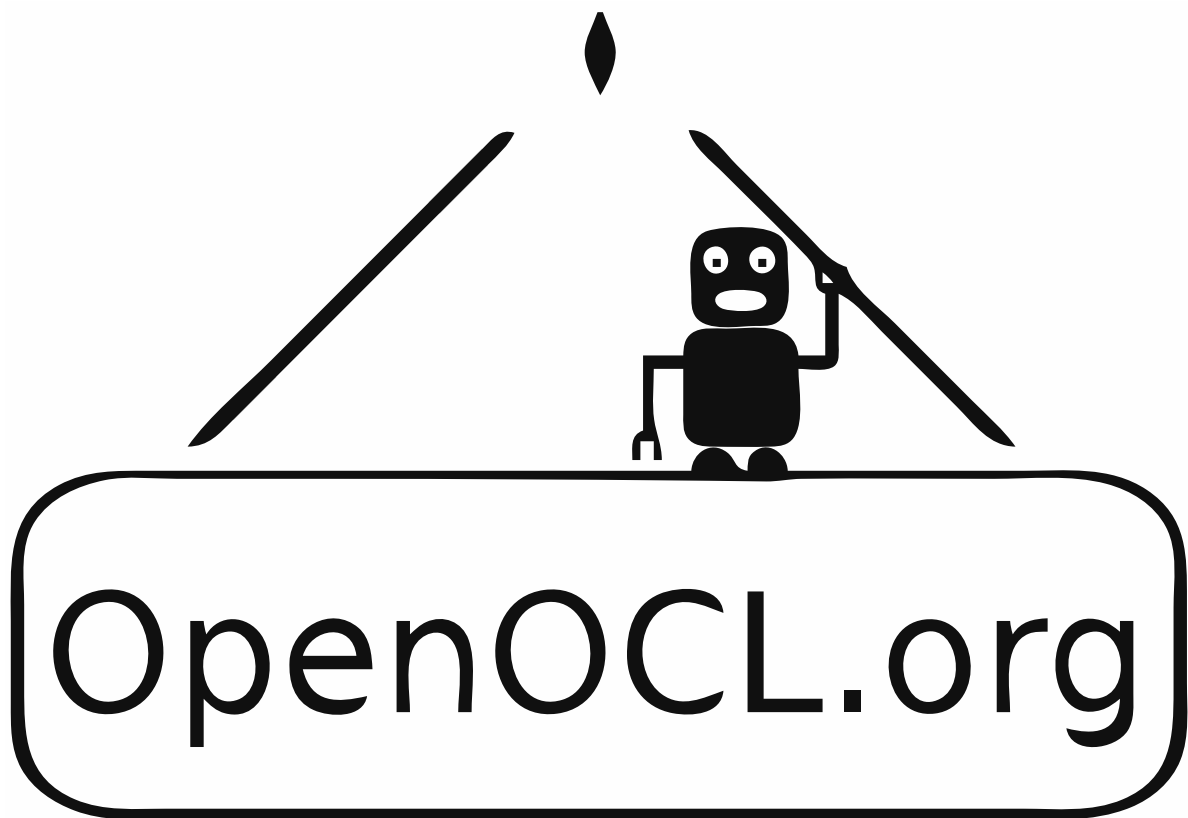




[View the Project on GitHub](#)

Versions: [v5](#) [v6](#) [v7](#)

API classes and modules

- [ocl.Constraint](#)
- [ocl.Cost](#)
- [ocl.DaeHandler](#)
- [ocl.MultiStageProblem](#)
- [ocl.Problem](#)
- [ocl.Simulator](#)
- [ocl.Stage](#)
- [ocl.Variable](#)
- [ocl.VarsHandler](#)
- [@\(conh,k,K,x,p\)](#)
- [@\(costh,k,K,x,p\)](#)
- [@\(costh,x,z,u,p\)](#)
- [@\(daeh,x,z,u,p\)](#)
- [@\(vh\)](#)
- [CasadiOptions](#)

[back](#)[top](#)

Documentation (OpenOCL v7)

This is the documentation of the current version *OpenOCL v7*.

In *OpenOCL* you can solve a large class of optimal control problems including non-linear, continuous-time, multi-stage, and constrained problems, which can appear in the context of **trajectory optimization** and **model predictive control**. The types of dynamical systems that are supported are all systems that can be described by **ordinary differential equations** or **differential algebraic equations**. 微分代数方程组

In the following we give a short introduction to the concepts used and introduced in *OpenOCL*. Scroll down or use the navigation on the left to look for specific classes and functions.

Cost terms and constraints

We support **bounds** that hold along the entire trajectory, and **grid-constraints** that hold only at specific gridpoints τ_k in the discretized trajectory. You can specify the gridpoints by using the argument **N** for in [ocl.Problem](#).

For the cost terms you can specify **path-costs** that are integrated along the trajectory (also known as Lagrange term), and **grid-costs** that can be specified for specific points on the discretized trajectory.

Optimal control problem definition (single-stage)

For optimal control problems with a single stage (most problems), *OpenOCL* supports the following type of optimal control problems:

$$\min_{x,z,u,p,T} \int_0^T L(x(t), z(t), u(t), p) dt + \Phi(x(T), p) + \sum_{k=1}^{N+1} \Psi_k(x(\tau_k), p) \quad \text{top}$$

subject to

$$\frac{d}{dt}x(t) = f(x(t), z(t), u(t), p) \quad \text{for } t \in [0, T]$$

$$0 = g(x(t), z(t), u(t), p) \quad \text{for } t \in [0, T]$$

$$r_k(x(\tau_k), p) \leq 0 \quad \text{for } k \in \{1, \dots, N+1\}$$

$$\underline{x}_0 \leq x(0) \leq \bar{x}_0$$

$$\underline{x}_e \leq x(T) \leq \bar{x}_e$$

$$\underline{x} \leq x(t) \leq \bar{x} \quad \text{for } t \in [0, T]$$

$$\underline{z} \leq z(t) \leq \bar{z} \quad \text{for } t \in [0, T]$$

$$\underline{u}_k \leq u(t) \leq \bar{u}_k \quad \text{for } t \in [\tau_k, \tau_{k+1}], \forall k \in \{1, \dots, N\}$$

$$\underline{p} \leq p \leq \bar{p}$$

$$T \in \mathbb{R}^+ \quad (\text{or } T = \hat{T} \text{ for a constant end time})$$

$$x : \mathbb{R} \rightarrow \mathbb{R}^{n_x}, \quad z : \mathbb{R} \rightarrow \mathbb{R}^{n_z}, \quad u : \mathbb{R} \rightarrow \mathbb{R}^{n_u}, \quad p \in \mathbb{R}^{n_p}$$

where $x(t)$ is the state trajectory, $u(t)$ the control trajectory, $z(t)$ the algebraic state trajectory, p the parameters, $L(x, z, u, p)$ the path cost function, $\Phi(x, p)$ the terminal cost function, $\Psi_k(x, p)$ the grid-cost function, $f(x, z, u, p)$ the system dynamics function (differential equation), $g(x, z, u, p)$ the algebraic constraints function, $r_k(x, p)$ the grid-constraints function. Note that the control bounds can be specified of each control interval, $\underline{u}_k, \bar{u}_k$.

If no algebraic states z are defined, the system is described by an *ordinary differential equation*. If algebraic states are defined, then the dynamics function $f(x, z, u, p)$ together with the algebraic constraints function $g(x, z, u, p)$ define the system dynamics as a *differential algebraic equation* in the semi-implicit form. To solve an optimal control problem, the *index* of the differential algebraic equation has to be smaller than or equal to one.

The dimension of the variables are: number of states n_x , number of algebraic states n_z , number of controls n_u , number of parameters n_p .

Single stage optimal control problems are implemented using the class [ocl.Problem](#).

Multi-stage optimal control problems

Multi-stage problems (also called multi-phase problems) allow you to **define a different system dynamics function for each of the stages**. This formulation also [top](#) allows you to implement discrete *events* where a jump in the state trajectory happens.

$$\min_{\substack{x_1, \dots, x_M \\ z_1, \dots, z_M \\ u_1, \dots, u_M \\ p_1, \dots, p_M \\ T_1, \dots, T_M}} \sum_{i=1}^M \left(\int_{T_{i-1}}^{T_i} L_i(x_i(t), z_i(t), u_i(t), p_i) dt + \Phi_i(x_i(T_i), p_i) + \sum_{k=1}^{N_i+1} \Psi_{i,k}(x_i(\tau_{i,k}), p_i) \right)$$

subject to

for $i \in \{1, \dots, M\}$:

$$\frac{d}{dt} x_i(t) = f_i(x_i(t), z_i(t), u_i(t), p_i) \quad \text{for } t \in [T_{i-1}, T_i]$$

$$0 = g_i(x_i(t), z_i(t), u_i(t), p_i) \quad \text{for } t \in [T_{i-1}, T_i]$$

$$r_{i,k}(x_i(\tau_{i,k}), p_i) \leq 0 \quad \text{for } k \in \{1, \dots, N_i+1\}$$

$$\underline{x}_{i,0} \leq x_i(T_{i-1}) \leq \bar{x}_{i,0}$$

$$\underline{x}_{i,e} \leq x_i(T_i) \leq \bar{x}_{i,e}$$

$$\underline{x}_i \leq x(t) \leq \bar{x}_i \quad \text{for } t \in [T_{i-1}, T_i]$$

$$\underline{z}_i \leq z(t) \leq \bar{z}_i \quad \text{for } t \in [T_{i-1}, T_i]$$

$$\underline{u}_{i,k} \leq u(t) \leq \bar{u}_{i,k} \quad \text{for } t \in [\tau_{i,k}, \tau_{i,k+1}], \forall k \in \{1, \dots, N_i\}$$

$$\underline{p}_i \leq p_i \leq \bar{p}_i$$

$$T_i \in \mathbb{R}^+ \quad (\text{or } T_i = \hat{T}_i \text{ for a constant end time})$$

for $i \in \{1, \dots, M-1\}$:

$$\phi_i(x_{i+1}(T_i), x_i(T_i)) = 0$$

$$x_i : \mathbb{R} \rightarrow \mathbb{R}^{n_x, i}, \quad z_i : \mathbb{R} \rightarrow \mathbb{R}^{n_z, i}, \quad u_i : \mathbb{R} \rightarrow \mathbb{R}^{n_u, i}, \quad p_i \in \mathbb{R}^{n_p, i}$$

where M is the number of stages, and ϕ_i is the stage transition function that relates the first state x_{i+1} of stage $i+1$ to the last state x_i of stage i (at timepoint T_i).

All other functions and bounds can be specified for each stage as described above in the single-stage formulation. Stages are implemented using the class [ocl.Stage](#), and the complete multi-stage problem can then be implemented using the class [ocl.MultiStageProblem](#).

ocl.Constraint()

The constraint handler allows to add constraints to the optimal control problem definition.

[top](#)

Methods

add(*lhs*, *op*, *rhs*)

Adds a constraint to the optimal control problem.

Arguments:

lhs ([ocl.Variable](#) or Matlab matrix) Left hand side of the constraint equation

op (char) One of the following operators as a string: '<=', '==', '>='

rhs ([ocl.Variable](#) or Matlab matrix) Right hand side of the constraint equation

userdata()

Returns the userdata object that can be passed to [ocl.Problem](#) or [ocl.Stage](#).

ocl.Cost()

The cost handler allows to add cost terms in a cost function definition.

Methods

add(*cost*)

Adds a cost term.

Arguments:

cost ([ocl.Variable](#) or Matlab matrix) Scalar variable containing the cost

userdata()

Returns the userdata object that can be passed to [ocl.Problem](#) or [ocl.Stage](#).

ocl.DaeHandler()

The differential equations handler allows to specify the system equations which can be of ODE and DAE type.

Methods

setODE(*id*, *equation*)

Adds a differential equation to the system. Note that for every state variable defined in the variables function, a differential equation must

be specified.

[top](#)

Arguments:

id (string) Name of the state variable for that the differential equation is given.

equation ([ocl.Variable](#) or Matlab matrix) The equation specifies the derivative of a state variable. Right hand side of the differential equation $\dot{x} = f(x,z,u,p)$ for state variable x .

[setAlgEquation](#)(**equation**)

Adds an algebraic equation to the system. Note that in order to be able to simulate the system, the total number of rows of the algebraic equations needs to be equal to the total number/dimension of algebraic variables.

Arguments:

equation ([ocl.Variable](#) or Matlab matrix) Algebraic equation g in the form $g(x,z,u,p)=0$

[userdata\(\)](#)

Returns the userdata object that can be passed to [ocl.Problem](#) or [ocl.Stage](#).

[ocl.MultiStageProblem](#)(**stages, transitions, userdata**)

Creates a multi-stage optimal control problem (multiple phases).

Arguments:

stages (cell<[ocl.Stage](#)>) List (cell-array) of stages.

transitions (cell<[ocl.Transition](#)>) List (cell-array) of transitions. The number of transitions has to be one less than the number of stages.

userdata (Any type, for example a struct or list.) A data field that can be used to pass any kind of constant data to the model functions. The userdata can be accessed by using the userdata property of `ocl.Cost`, `ocl.Constraint`, `ocl.DaeHandler`, and `ocl.VarHandler`. Defaults to an empty list.

Returns:

([ocl.MultiStageProblem](#)) The problem object.

Methods

[top](#)

getInitialGuess()

Use this method to retrieve a first initial guess that is generated from the bounds. You can further modify this initial guess to improve the solver performance.

Returns:

(cell<OclVariable>) Structured variable for setting the initial guess for each stage (given as cell array).

solve(initialGuess)

Calls the solver and starts doing iterations.

Arguments:

initialGuess (cell<OclVariable>) Structured variable containing the initial guess for each stage (given as cell array).

Returns:

(cell<OclVariable>) The solution of the OCP for each stage (given as cell array).

(cell<OclVariable>) Time points of the solution

ocl.Problem(T, vars, dae, pathcosts, gridcosts, gridconstraints, terminalcost, casadi_options, N, d, verbose, userdata, nlp_casadi_mx, controls_regularization, controls_regularization_value)

↓
Number of control intervals

Creates a single-stage optimal control problem.

```

function vanderpol
    ↑ 运行时长
    problem = ocl.Problem(10, @varsfun, @daefun, @pathcost, 'N',

    problem.setInitialBounds('x', 0);
    problem.setInitialBounds('y', 1);

    problem.initialize('x', [0 1], [-0.2 -0.2]);

    [solution, timepoints] = problem.solve();

    % plotting of control and state p trajectory:
    ocl.plot(timepoints.controls, solution.controls.u)
    ocl.plot(timepoints.states, solution.states.p)
end
  
```

```

function varsfun(vh)
    vh.addState('x', 'lb', -0.25, 'ub', inf);
    vh.addState('y');
    vh.addControl('F', 'lb', -1, 'ub', 1);
end

function daefun(daeh,x,z,u,p)
    daeh.setODE('x', (1-x.y^2)*x.x - x.y + u.F);
    daeh.setODE('y', x.x);
end

function pathcost(ch,x,z,u,p)
    ch.add( x.x^2 );
    ch.add( x.y^2 );
    ch.add( u.F^2 );
end

```

Van der Pol oscillator SAE

$$\begin{cases} \dot{x} = \mu(1-y^2)x - y \\ \dot{y} = x \end{cases}$$

Arguments:

- T** (numeric or []) The end time/horizon length of the optimal control problem. If your system equations are expressed as function of an independent variable other than time, T represents not the end time but the endpoint of the integration over the independent variable. If you would like to optimize for time in a **time optimal control** formulation pass the empty list [].
- vars** (@(vh)) System variables function. Defaults to an empty function handle.
- dae** (@(daeh,x,z,u,p)) DAE (system equations) function. Defaults to an empty function handle.
- pathcosts** (@(costh,x,z,u,p)) Path-cost function. Defaults to a function handle returning 0.
- gridcosts** (@(costh,k,K,x,p)) Grid-cost function. Defaults to a function handle returning 0.
- gridconstraints** (@(conh,k,K,x,p)) Grid-constraints function. Defaults to no constraints added.
- terminalcost** (@(ch,x,p)) Terminal cost function. Defaults to a function handle returning 0.
- casadi_options** (CasadiOptions) Options struct for casadi and ipopt.
- N** (Integer) Number of control intervals. Defaults to 20.
- d** (Integer in [2,5]) Degree of the interpolating polynomial in each control interval. Defaults to 3.

verbose (Boolean) If set to `false` no solver output will be shown. Defaults to `true`.

[top](#)

userdata (Any type, for example a struct or list.) A data field that can be used to pass any kind of constant data to the model functions. The `userdata` can be accessed by using the `userdata` property of [ocl.Cost](#), [ocl.Constraint](#), [ocl.DaeHandler](#), and [ocl.VarHandler](#). Defaults to an empty list.

nlp_casadi_mx (Boolean) If set to `true`, `casadi.MX` symbolics are used instead of `casadi.SX` symbolics. Defaults to `false`.

controls_regularization (Boolean) If set to `true`, a small cost on the control inputs is added depending on the weight given by `controls_regularization_value`. Defaults to `true`.

controls_regularization_value (Float) The weight for the `controls_regularization`. Defaults to `1e-6`.

Returns:

(ocl.Problem) The Problem object.

Methods

solve()

Calls the solver and starts doing iterations.

Returns:

(ocl.Variable) The solution of the OCP
(ocl.Variable) Grid points of the solution

solve(ig)

Calls the solver and starts doing iterations.

Arguments:

ig (ocl.Variable) An initial guess, for example from a previous solution.

Returns:

(ocl.Variable) The solution of the OCP
(ocl.Variable) Grid points of the solution

setBounds(id, lb, ub)

Sets a bound on a variable for the whole trajectory. If only the lower bound is given, it will be `lb==ub`.

Arguments:

id (string) The variable id
lb (numeric) The lower bound
ub (numeric,optional) The upper bound

[top](#)

setInitialBounds(id, lb, ub)

Sets an initial bound on a variable. If only the lower bound is given, it will be $lb == ub$.

Arguments:

id (string) The variable id
lb (numeric) The lower bound
ub (numeric,optional) The upper bound

setEndBounds(id, lb, ub)

Sets an end bound on a variable. If only the lower bound is given, it will be $lb == ub$.

Arguments:

id (string) The variable id
lb (numeric) The lower bound
ub (numeric,optional) The upper bound

initialize(id, gridpoints, values)

Sets an initial guess for a variable.

Arguments:

id (string) The variable id
gridpoints (numeric) The normalized gridpoints. The gridpoints is a list of values between 0 and 1 where 0 is the beginning of the trajectory and 1 is the final time of the trajectory.
values (numeric) The initial guess values at the gridpoints. The number of columns of values must be equal to the length of gridpoints.

ocl.Simulator(vars, dae)

Performs simulations of the system by integrating the system dynamics.

Arguments:

vars ([@\(vars_handler\)](#)) System variables function. Optional, defaults to an empty function handle.

[top](#)

dae ([@\(dae_handler,x,z,u,p\)](#)) DAE (system equations) function. Optional, defaults to an empty function handle.

Returns:

([Simulator](#)) the Simulator object.

Methods**getStates()**

Returns a structured state variable that you can use to pass as the initial state to the simulator. All variable values default to zero.

Returns:

([ocl.Variable](#)) The states.

getParameters()

Returns a structured state variable that allows you to set parameters for the simulation. All parameter values default to zero.

Returns:

([ocl.Variable](#)) The parameters.

reset(x0)

Sets the initial state of the simulation.

Arguments:

x0 ([ocl.Variable](#) or numeric) Initial state

step(u, dt)

Simulate one step using the control input.

Arguments:

u ([ocl.Variable](#) or numeric) The controls

dt (numeric) The timestep

Returns:

([ocl.Variable](#)) The state.

([ocl.Variable](#)) Algebraic variables.

ocl.Stage(T, vars, dae, pathcosts, gridcosts, gridconstraints, terminalcost, N, d, userdata)

The definition of a Stage.

[top](#)

```
stage = ocl.Stage([], @vars, @ode, 'N', 10, 'd', 2);

% Function definitions can be in the same file
% (if the main script is wrapped by a function)
% or in separate files:
function vars(sh)
    sh.addState('s');
    sh.addState('v');
end

function ode(sh,x,~,~,~)
    sh.setODE('s', x.v);
    sh.setODE('v', -10);
end
```

Arguments:

- T** (numeric or []) The end time/horizon length of the optimal control problem. If your system equations are expressed as function of an independent variable other than time, T represents not the end time but the endpoint of the integration over the independent variable. If you would like to optimize for time, **time optimal control**, pass the empty list [].
- vars** (@(vh)) System variables function. Defaults to an empty function handle.
- dae** (@(daeh,x,z,u,p)) DAE (system equations) function. Defaults to an empty function handle.
- pathcosts** (@(costh,x,z,u,p)) Path-cost function. Defaults to a function handle returning 0.
- gridcosts** (@(costh,k,K,x,p)) Grid-cost function. Defaults to a function handle returning 0.
- gridconstraints** (@(conh,k,K,x,p)) Grid-constraints function. Defaults to no constraints added.
- terminalcost** (@(ch,x,p)) Terminal cost function. Defaults to a function handle returning 0.
- N** (Integer) Number of control intervals. Defaults to 20.
- d** (Integer in [2,5]) Degree of the interpolating polynomial in each control interval. Defaults to 3.
- userdata** (Any type, for example a struct or list.) A data field that can be used to pass any kind of constant data to the model functions. The userdata can be accessed by using the userdata property of ocl.Cost,

`ocl.Constraint`, `ocl.DaeHandler`, and `ocl.VarHandler`. Defaults to an empty list.

[top](#)

Methods

`setBounds(id, lb, ub)`

Sets a bound on a variable for the whole trajectory. If only the lower bound is given, it will be $lb==ub$.

Arguments:

- id** (string) The variable id
- lb** (numeric) The lower bound
- ub** (numeric,optional) The upper bound

`setInitialBounds(id, lb, ub)`

Sets an initial bound on a variable. If only the lower bound is given, it will be $lb==ub$.

Arguments:

- id** (string) The variable id
- lb** (numeric) The lower bound
- ub** (numeric,optional) The upper bound

`setEndBounds(id, lb, ub)`

Sets an end bound on a variable. If only the lower bound is given, it will be $lb==ub$.

Arguments:

- id** (string) The variable id
- lb** (numeric) The lower bound
- ub** (numeric,optional) The upper bound

`initialize(id, gridpoints, values)`

Sets an initial guess for a variable.

Arguments:

- id** (string) The variable id
- gridpoints** (numeric) The normalized gridpoints. The gridpoints is a list of values between 0 and 1 where 0 is the beginning of the trajectory and 1 is the final time of the trajectory.
- values** (numeric) The initial guess values at the gridpoints. The number of columns of `values` must be equal to the length of `gridpoints`.

ocl.Variable()

[top](#)

The ocl.Variable type is the basic structure to retrieve, store, modify structured optimization variables. You can access subvariables by their name like the state trajectory or the control variables.

```
% v is a solution of an OCP
% p=[px;py;pz] is of size 3x1
% p trajectory is of size 3x(N+1)
% F trajectory is of size 1xN
% with N control intervals
p = v.states.p;      % get state p trajectory
F = v.controls.F;    % get control F trajectory

% set all 3x1 p states to the same value
v.states.p = [3;2;1];
% set p states 4 and 5 in the trajectory
v.states.p(:,4:5) = [1,2,3;4,5,6].';

% set all px values of p in state trajectory
v.states.p(1,:) = 4;

% plotting of state p trajectory:
plot(t.states.value,v.states.p.value)
```

Methods

get(id)

Alternative syntax: var.**id** Gets a sub-variable of a variable. You can use the shorthand notation with the dot operator, e.g.: solution.states.x

Arguments:

id (string) Name of the state variable

Returns:

(ocl.Variable) the sub-variable of the given variable.

set(value)

Alternative syntax: var = **value** Sets a value to the variable.

Arguments:

value (numeric) The value to be set. The value either has to be of the same dimension as the variable or if possible it will be

The variables handler allows to specify the system variables, its dimensions and bounds.

[top](#)

Methods

addState(*id*, *s*, *lb=-inf*, *ub=inf*)

Adds a state variable to the system.

Arguments:

- id** (char) Name of the state variable
- s** (int, optional) Size of the state variable. Scalar, vector, and matrix valued variables are allowed. If a scalar value *s* is given, the size of the variable will be $[s,1]$. Defaults to $[1,1]$.
- lb=-inf** (numeric, optional) Lower bound on the variable. This value can be overwritten when you specify bounds for OclSolver with `solver.setBound`. Defaults to -inf.
- ub=inf** (numeric, optional) Upper bound on the variable. This value can be overwritten when you specify bounds for OclSolver with `solver.setBound`. Defaults to inf.

addAlgVar(*id*, *s*, *lb=-inf*, *ub=inf*)

Adds an algebraic variable to the system.

Arguments:

- id** (char) Name of the algebraic variable
- s** (int, optional) Size of the algebraic variable. Scalar, vector, and matrix valued variables are allowed. If a scalar value *s* is given, the size of the variable will be $[s,1]$. Defaults to $[1,1]$.
- lb=-inf** (numeric, optional) Lower bound on the variable. This value can be overwritten when you specify bounds for OclSolver with `solver.setBound`. Defaults to -inf.
- ub=inf** (numeric, optional) Upper bound on the variable. This value can be overwritten when you specify bounds for OclSolver with `solver.setBound`. Defaults to inf.

addControl(*id*, *s*, *lb=-inf*, *ub=inf*)

Adds an control input to the system.

Arguments:

- id** (char) Name of the control variable
- s** (int, optional) Size of the control variable. Scalar, vector, and matrix valued variables are allowed. If a scalar value *s* is given, the size of the variable will be $[s,1]$. Defaults to $[1,1]$.
- lb=-inf** (numeric, optional)

Lower bound on the variable. This value can be overwritten when you specify bounds for OclSolver with `solver.setBound`. Defaults to `-inf`.

ub=inf (numeric, optional) Upper bound on the variable. This value can be overwritten when you specify bounds for OclSolver with `solver.setBound`. Defaults to `inf`.

addParameter(id, s, default=[])

Adds a parameter.

Arguments:

id (char) Name of the parameter

s (int, optional) Size of the control variable. Scalar, vector, and matrix valued variables are allowed. If a scalar value `s` is given, the size of the variable will be `[s,1]`. Defaults to `[1,1]`.

default=[] (numeric, optional) Default value for the parameter. This value can be overwritten when you specify the parameter for OclSolver with `solver.setParameter`. Defaults to unbounded.

userdata()

Returns the userdata object that can be passed to `ocl.Problem` or `ocl.Stage`.

@(conh, k, K, x, p)

Function signature for the grid constraints function.

Arguments:

conh (`ocl.Constraint`) Constraints handler that allows to add constraints.

k (int) Current grid point index

K (int) Largest (last) grid point index

x (`ocl.Variable`) States

p (`ocl.Variable`) Parameters

@(costh, k, K, x, p)

Function signature for the grid cost function.

Arguments:

costh (`ocl.Cost`) Cost handler that allows to add cost terms.

k (int) Current grid point index

K (int) Largest (last) grid point index

x ([ocl.Variable](#)) States

p ([ocl.Variable](#)) Parameters

[top](#)

@(costh, x, z, u, p)

Function signature for the path cost function.

Arguments:

costh ([ocl.Cost](#)) Cost handler

x ([ocl.Variable](#)) State variables

z ([ocl.Variable](#)) Algebraic states

u ([ocl.Variable](#)) Controls

p ([ocl.Variable](#)) Parameters

@(daeh, x, z, u, p)

Function handle signature for DAE function.

Arguments:

daeh ([ocl.DaeHandler](#)) DAE handler

x ([ocl.Variable](#)) State variables

z ([ocl.Variable](#)) Algebraic states

u ([ocl.Variable](#)) Controls

p ([ocl.Variable](#)) Parameters

@(vh)

Function handle signature for system variables function.

Arguments:

vh ([ocl.VarsHandler](#)) Variables handler

CasadiOptions()

Creates an options struct for [ocl.Problem](#). Check the casadi documentation and the ipopt documentation to see which options are available. The ipopt options can be set in 'casadi_options.ipopt'. The default values are the following:

```
casadi_options = struct;  
casadi_options.ipopt = struct;  
casadi_options.ipopt.linear_solver = 'mumps';  
casadi_options.ipopt.hessian_approximation = 'exact';
```

Returns:

(struct) the options struct.

[top](#)